GA4GH-RegBot: An LLM-Powered Compliance Assistant for Genomic Data Sharing

Google Summer of Code Program 2026 Project Proposal

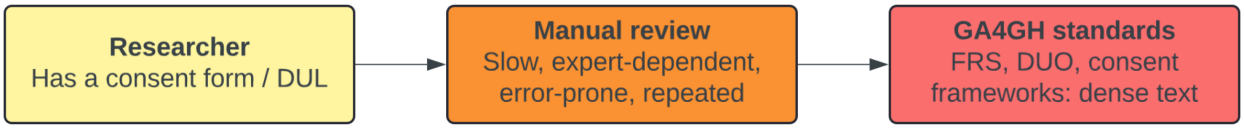
Khoa Minh Pham
minhpham181004@gmail.com
https://github.com/minhpham1810

Abstract. The core technical contribution of GA4GH-RegBot is a clause-level citation contract: every compliance verdict is grounded in a retrieved GA4GH clause, cross-checked by a post-generation validator that withholds any claim not traceable to the indexed corpus. This is what separates RegBot from a general-purpose chatbot: compliance tooling for genomic data sharing is one of the few RAG contexts where citation accuracy is genuinely non-negotiable, not a nice-to-have. GA4GH-RegBot automates the gap analysis between a researcher's uploaded document and the GA4GH policy corpus, removing a manual, error-prone process that most institutions repeat from scratch every time. Output takes a two-part form: a structured per-clause verdict table (covered / partially covered / missing) and a plain-language narrative summary, making the system's reasoning auditable at two levels of granularity. Built on open-source LLMs running fully locally, RegBot removes the data governance barrier that prevents researchers in many jurisdictions from using commercial API-based tools, lowering the barrier to responsible genomic data sharing globally. The target corpus is bounded: the Framework for Responsible Sharing of Genomic and Health-Related Data (FRS), the Data Use Ontology (DUO), and the Consent Clarity Toolkit (approximately 80,000–100,000 tokens of policy text) making high-precision clause-level retrieval achievable within the GSoC timeline.

Project Description

I. The Problem

Sharing genomic data responsibly is one of the central challenges in modern biomedical research. The GA4GH (Global Alliance for Genomics and Health) has produced a suite of policy frameworks including the Framework for Responsible Sharing of Genomic and Health-Related Data, the Data Use Ontology (DUO), and the Consent Clarity Toolkit to help researchers navigate this space. However, applying these frameworks to a specific study's consent form or data use letter (DUL) remains a deeply manual process. A researcher must read dense legal text, locate the applicable clause, compare it against their own language, and determine whether it passes or fails each requirement. This work is slow, requires specialised policy expertise, and is systematically repeated at every institution worldwide.



Before designing the system, I looked at what actually goes wrong in document-intensive RAG deployments. Three failure modes stood out as directly relevant to compliance tooling. First, when a required GA4GH clause is absent from the indexed corpus, a naive system will hallucinate a plausible-sounding policy rather than admitting the gap (missing content):

RegBot addresses this through the citation validator, which withholds any response not grounded in retrieved text. Second, even when the correct clause is retrieved, a model distracted by surrounding noise can fail to extract the obligation accurately (not extracted): RegBot addresses this by enforcing structured JSON output that forces the model to quote evidence explicitly. Third, a model may produce a verdict that is technically accurate but incomplete, missing obligations that were present in the retrieved context (incomplete): RegBot addresses this through the two-part output contract, requiring both a per-clause structured table and a narrative summary that must account for every retrieved obligation. **Result:** The adoption of GA4GH standards is slower than it should be, and the errors that do occur can have real consequences: data misuse, consent violations, or blocked cross-border data transfers.

II. The Solution: GA4GH-RegBot

GA4GH-RegBot is a RAG-powered compliance assistant that automates the gap analysis between a researcher's uploaded document and the GA4GH policy corpus. The researcher uploads their data use letter or consent form; the system retrieves the exact policy clauses most relevant to that document's language; an open-source LLM (Llama 3 or Mistral, running locally) then produces a two-part compliance report: a structured per-clause verdict (`covered` / `partially covered` / `missing`) paired with a narrative summary explaining each gap and citing the specific article and page number that supports it.

The core design principle, which is borrowed directly from production RAG systems, is that the LLM is never permitted to reason from general knowledge about genomic ethics. Every claim in its output must be grounded in a retrieved chunk from the GA4GH corpus. This clause-level citation requirement is what separates RegBot from a general-purpose chatbot and makes its output auditable. Importantly, compliance review is rarely single-turn: a researcher may revise their question, re-examine a specific clause, or re-upload an amended document. RegBot is therefore designed as a session-aware system, preserving conversation context so that each follow-up query is grounded in both prior exchanges and the original uploaded document.

III. System Architecture

The architecture has four main layers: ingestion, retrieval, generation and gap detection, and a Streamlit UI. There is also a lightweight off-topic guard that sits in front of the pipeline to handle irrelevant queries gracefully.

1. Ingestion layer. GA4GH policy documents are parsed, chunked at 1,000–1,200 characters with 200-character overlap, and stored in a persistent ChromaDB vector database. Each chunk carries structured metadata: `doc_type` (framework, guideline, or ontology term), `article_id`, `source_url`, and `page`. Researcher-uploaded documents (DULs, consent forms) undergo the same preprocessing and are embedded into a separate ephemeral collection tied to the active session, enabling direct semantic comparison with policy chunks.

2. Retrieval layer. When a researcher submits a DUL, `classifier.py` first determines which GA4GH framework domain is most relevant: consent, data access, or cross-border transfer. The system accepts researcher-uploaded documents in PDF and plain text (`.txt`) formats. Scanned or image-based PDFs are out of scope for the initial version, as institutional DULs and consent forms are typically born-digital documents. `decomposer.py` handles query rewriting for follow-up questions, producing a standalone query from the

conversation context. For complex multi-clause DULs, sub-query decomposition into per-clause queries is added as a Milestone 2 enhancement once the core single-query pipeline is validated. `retriever.py` runs MMR semantic search over ChromaDB for each sub-query, filtered by `doc_type` and `article_id`, returning a diverse but clause-precise set of chunks. As a Milestone 2 enhancement, the retrieval layer will extend to hybrid search, combining BM25 keyword matching with dense semantic search via LangChain's `EnsembleRetriever` and Reciprocal Rank Fusion, followed by a `BAAI/bge-reranker-base` cross-encoder reranker. This is particularly important for GA4GH compliance, where legal terminology must be matched exactly, and semantic search alone can smooth over distinctions that matter for compliance verdicts.

3. Generation and gap detection layer. Retrieved chunks are assembled into a `KNOWLEDGE` block and passed to the LLM with a strict system prompt. The prompt instructs the model to produce a two-part output: first, a structured JSON compliance table mapping each retrieved obligation to a `covered` / `partially covered` / `missing` verdict with a direct quote from the researcher's document; second, a narrative summary that explains each gap in plain language and cites the exact GA4GH clause (`article_id`, `page`) that was not satisfied. A post-generation validator cross-checks every cited `article_id` against the retrieved set: any claim citing a clause outside the retrieved context is flagged as unverified and withheld. The system prompt is stored as a `.md` file and loaded at import time, separating prompt iteration from application logic, updating compliance instructions as GA4GH frameworks evolve requires only editing the markdown file, not modifying Python source.

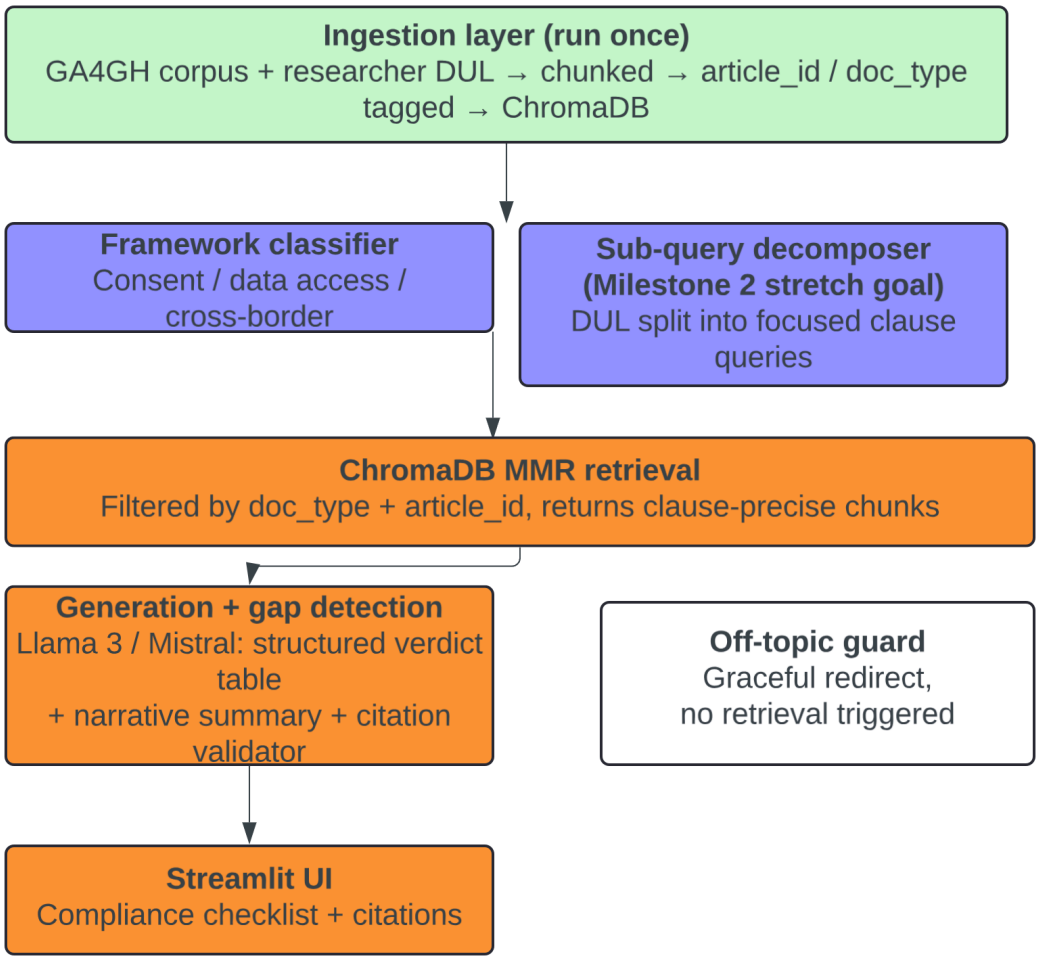
4. Presentation layer. A Streamlit UI renders the two-part report as an interactive compliance checklist. Each row in the structured table is expandable to show the cited GA4GH passage and the researcher's matching (or missing) text. The narrative summary appears below.

5. Optional interaction layer. A lightweight model handles general-purpose or out-of-scope queries. Rather than failing silently, the system responds gracefully and redirects the researcher toward the compliance-focused workflow. This layer keeps the main pipeline computationally lean by intercepting irrelevant queries before they reach the retrieval and generation stages.

Why This Architecture: Most of these design decisions came from reading about what actually works in production RAG systems: metadata filtering, MMR search, session-aware context and asking whether each one had a clear compliance-specific reason to exist. They do. The one novel addition is the two-part output contract: requiring both a structured per-clause verdict and a narrative summary means the system's reasoning is auditable at two levels of granularity, which is appropriate for a tool whose outputs inform real data governance decisions. The choice of fully local inference via Ollama is deliberate: genomic consent data and institutional DULs may be sensitive, and researchers in many jurisdictions cannot send them to a commercial API endpoint. Running inference locally removes that barrier entirely, making the tool deployable without institutional data governance review.

Corpus scope: The GA4GH policy corpus ingested by RegBot consists of three documents: the FRS PDF (~75 pages), the DUO ontology exported as structured text (~300 terms with definitions), and the Consent Clarity Toolkit (~25 pages). Combined, this is approximately 80,000–100,000 tokens — large enough to require retrieval rather than context-stuffing, but small enough that a cross-encoder reranker is CPU-feasible and that a hand-labelled evaluation set can be built exhaustively from the clause structure.

Scope boundaries: A few things are intentionally out of scope for this version. RegBot will not attempt multi-hop reasoning across frameworks: each DUL is assessed against one framework domain at a time. There is no ontology mapping in v1; DUO terms are retrieved as text chunks, not traversed as a graph. OCR support for scanned documents is excluded from the core build. These are not gaps in the design, they are deliberate decisions to keep the 175-hour scope executable.



Example Codebase Structure

```
None

ga4gh-regbot/
├── ingestion/
│   ├── ingest.py              # Main ingestion entrypoint
│   ├── chunker.py             # Semantic chunking + article_id tagging
│   ├── metadata.py            # doc_type / article_id / source_url schema
│   └── loaders.py             # PDF + web page loaders (PyMuPDF, requests)
```

```
├── retrieval/
│   ├── retriever.py           # MMR search with doc_type / article_id filters
│   ├── classifier.py          # Framework domain classifier
│   │                            (consent / data access / cross-border)
│   └── decomposer.py          # Sub-query decomposition for multi-clause DULs
│                                (Milestone 2 stretch goal)
├── generation/
│   ├── pipeline.py        # End-to-end RAG orchestration + off-topic guard
│   ├── prompts/
│   │   ├── __init__.py    # Loads .md files at import time
│   │   └── system.md      # System prompt — grounding contract + output format
│   ├── gap_detector.py    # Structured verdict table + narrative summary
│   └── validator.py       # Post-generation citation validator
├── ui/
│   └── app.py                 # Streamlit interface
├── evaluation/
│   ├── test_retrieval.py      # Clause lookup evaluation against labelled set
│   └── test_citations.py      # Citation accuracy + false-positive rate
├── data/
│   ├── ga4gh_corpus/          # Raw GA4GH PDFs and web exports
│   └── test_duls/             # Sample DULs for evaluation
├── chroma_db/                 # Persisted vector store
├── config.py                  # Model, chunking, and retrieval settings
└── requirements.txt
```

The directory mirrors the pipeline layers directly: each layer is a self-contained module with no circular dependencies. Swapping ChromaDB for FAISS requires changes only inside `retrieval/`. Replacing the LLM backend requires changes only inside `generation/`. The `metadata.py` schema is the load-bearing element of the entire pipeline: the `doc_type` and `article_id` fields it defines are what make filtered retrieval, structured verdict output, and citation validation all work consistently.

Code Sketches

I want to highlight two parts of the implementation that took the most thought to get right. The chunker is where metadata gets attached to each chunk: if `article_id` extraction is wrong here, everything downstream breaks. The validator is the enforcement mechanism that makes the citation contract real rather than just a prompt instruction.

1. `ingestion/chunker.py`: metadata-tagged chunking

```python
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_core.documents import Document
import re

ARTICLE_RE = re.compile(r'(?i)(section|article|clause)\s+([\d\.]+)')

def chunk_document(raw_docs: list[Document], doc_type: str, source_url: str) ->
 list[Document]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1200, chunk_overlap=200
    )
    chunks = splitter.split_documents(raw_docs)
    for chunk in chunks:
        match = ARTICLE_RE.search(chunk.page_content)
        chunk.metadata.update({
            "doc_type":   doc_type, # e.g. "framework", "ontology"
            "article_id": match.group(2) if match else None, # e.g. "3.2.1",
 "DUO:0000045"
            "source_url": source_url,
            "page":       chunk.metadata.get("page", 0) + 1, # 1-indexed
        })
    return chunks
```

2. `generation/validator.py`: post-generation citation validator

```python
import json

def validate_citations(llm_response: str, retrieved_ids: set[str]) ->
 tuple[str, list[str]]:
    flagged = []
    try:
        start = llm_response.find("[")
```

```
        end   = llm_response.rfind("]") + 1
        table = json.loads(llm_response[start:end])
        for row in table:
            aid = row.get("article_id")
            if aid and aid not in retrieved_ids:
                flagged.append(aid)
                row["status"]   = "unverified"
                row["evidence"] = None
        return json.dumps(table), flagged
    except Exception:
        return llm_response, flagged
```

End-to-end Example

To make this concrete, here is what actually happens when a researcher uploads a DUL.

Input: researcher's uploaded data use letter (excerpt)

> *"Data generated under this agreement may be used for the primary research purpose described in the approved protocol. Re-identification of participants is strictly prohibited. Data will be stored on institutionally managed servers and deleted upon project completion. Secondary use by third parties is not addressed in this agreement."*

`retrieval/classifier.py` identifies the domain as `consent + data_access`. **`retrieval/retriever.py`** runs MMR search filtered by `doc_type: framework` and `doc_type: ontology`, returning among others:

`article_id`	Retrieved clause
`FRS-3.2.1`	*"Data use must be limited to the purposes disclosed to and authorised by the data subject at the time of consent, including any explicitly permitted secondary uses."*
`FRS-4.1`	*"Data controllers must document retention schedules and ensure data are not retained beyond the period necessary for the authorised purpose."*
`DUO:0000007`	*"Disease-specific use: data may only be*

	used for research into a specific disease or condition."

`generation/gap_detector.py` passes these chunks as the `KNOWLEDGE` block to the LLM. The system prompt enforces two-part output. **`generation/validator.py`** cross-checks every `article_id` in the response against `{"FRS-3.2.1", "FRS-4.1", "DUO:0000007"}` all pass.

Structured verdict table (`gap_detector.py` output)

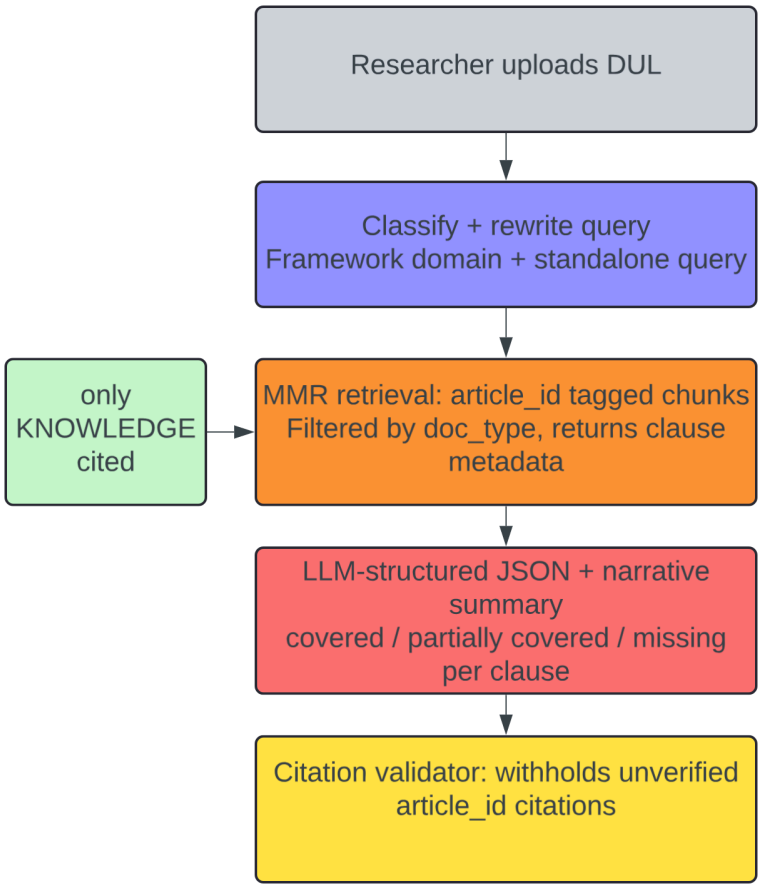
```JSON
[
  {
    "article_id": "FRS-3.2.1",
    "obligation": "Secondary use must be explicitly addressed and authorised",
    "status": "missing",
    "evidence": null
  },
  {
    "article_id": "FRS-4.1",
    "obligation": "Retention schedule must be documented with defined
 timelines",
    "status": "partially covered",
    "evidence": "Data will be stored on institutionally managed servers and
 deleted upon project completion."
  },
  {
    "article_id": "DUO:0000007",
    "obligation": "Disease-specific use scope must be declared",
    "status": "missing",
    "evidence": null
  }
]
```

Narrative summary (`gap_detector.py` output)

> *"Your data use letter has two critical gaps relative to GA4GH requirements. First, it does not address secondary use by third parties (FRS-3.2.1) — GA4GH requires that any permitted secondary uses be explicitly stated and authorised at the time of consent. Second, while deletion upon project completion is mentioned, there is no documented retention schedule with defined timelines (FRS-4.1). Additionally, the disease-specific scope of data use is not declared, which is required under*

DUO:0000007 if data use is restricted to a particular condition. These gaps should be resolved before submission for data access approval."

`ui/app.py` renders the verdict table as an interactive compliance checklist. Each row is expandable to show the fully-retrieved GA4GH clause alongside the researcher's matching or missing text. The narrative summary appears below. Because session context is preserved in `st.session_state`, the researcher can then ask a follow-up — *"What language should I add to address FRS-3.2.1?"* — without re-uploading the document, and the pipeline reuses the existing session context.



Technology Stack

Layer	Tool / Library	Rationale
LLM orchestration	LangChain	Mature RAG pipeline primitives; supports metadata filtering natively
Language model	Llama 3 8B / Mistral 7B via Ollama	Fully local, no sensitive DUL data sent to external APIs

Embeddings	`sentence-transformers` (`all-MiniLM-L6-v2` or similar)	Open-source, no API key required
Vector Database database	ChromaDB	Persistent, file-based, supports metadata filtering by `doc_type` and `article_id`
Hybrid search	`rank_bm25` + LangChain `EnsembleRetriever`	Catches exact legal term matches that semantic search smooths over; combined via Reciprocal Rank Fusion
Reranker	`BAAI/bge-reranker-base` (HuggingFace)	Cross-encoder reranking for clause-level precision; CPU-feasible on the small GA4GH corpus
Document parsing	PyMuPDF / `pypdf`	Handles researcher uploads reliably
UI	Streamlit	Rapid prototyping; supports file upload and structured table rendering

IV. Project Deliverables

Proposed Schedule

March 16 – March 31, 2026 (Application Period)

I am using this period to finish the proposal and get familiar with the GA4GH corpus. I have already cloned the relevant GA4GH repositories, installed ChromaDB and Ollama locally, and run a proof-of-concept ingestion of the FRS PDF to validate that `article_id` metadata can be reliably extracted via regex. I am also engaging with the Slack channel to clarify scope questions around compliance gap output format and DUO integration.

April 1 – April 30, 2026 (Acceptance Waiting Period)

I will complete a full manual annotation of 30–40 GA4GH clause lookups to serve as the retrieval evaluation ground truth, begin structuring the `ingestion/` module, and read the LangChain documentation on metadata filtering and `EnsembleRetriever`. I will also make my first commits to a public GitHub repository so that code is traceable from day one.

May 1 – May 24, 2026 (Community Bonding Period)

I will use this period to finalise the development environment with my mentor, agree on the exact evaluation metrics and acceptance thresholds for Milestone 1, confirm the full list of

GA4GH source documents to be ingested, and set up the GitHub repository with branch protection, a `CONTRIBUTING.md`, and a CI workflow for running the evaluation test harnesses. I will also share the proof-of-concept ingestion I have already completed during the application period, including the populated ChromaDB collection and the `article_id` extraction results for the FRS PDF, so my mentor has a concrete artifact to evaluate before coding officially begins. I will schedule weekly check-ins with Dedy and confirm the communication channels (Slack, GitHub issues) for the coding period.

May 25 – July 6, 2026 (Standard Coding Period — Milestone 1)

This period covers the full ingestion and retrieval layer. Work proceeds in weekly increments with daily commits to the public GitHub repository.

- Week 1 (May 25–31): Implement `ingestion/loaders.py` — PDF and web page loading via PyMuPDF and `requests`.
- Week 2 (Jun 1–7): Implement `ingestion/chunker.py` — 1,200-character chunking with 200-character overlap, `article_id` regex extraction, `doc_type` and `source_url` tagging, with unit tests.
- Week 3 (Jun 8–14): Implement `ingestion/ingest.py` — orchestrate full ingestion of FRS, DUO, and Consent Clarity Toolkit into ChromaDB.
- Week 4 (Jun 15–21): Implement `retrieval/classifier.py` and `retrieval/retriever.py` — framework domain classifier and MMR search with metadata filters.
- Week 5 (Jun 22–28): Build `evaluation/test_retrieval.py` — run retrieval evaluation against the hand-labelled clause lookup set, measure MRR and Recall@K.
- Week 6 (Jun 29–Jul 6): Fix retrieval issues surfaced by evaluation, write ingestion documentation, prepare midterm presentation slides.

Success Criteria: We consider retrieval validated if Recall@5 ≥ 0.80 against the hand-labelled set.

Midterm deliverables (July 6–10): Fully functional ingestion pipeline for the complete GA4GH corpus, working MMR retrieval with metadata filtering, and a retrieval evaluation report with MRR and Recall@K scores. All code committed to GitHub with passing tests.

July 6 – August 24, 2026 (Work Period: Milestone 2 + Milestone 3)

This period covers generation, gap detection, citation validation, the Streamlit UI, and documentation.

- Week 7 (Jul 7–13): Implement `generation/prompts/` — system prompt enforcing two-part output contract. Integrate Llama 3 8B via Ollama.
- Week 8 (Jul 14–20): Implement `generation/gap_detector.py` — structured verdict table with `covered` / `partially covered` / `missing` per clause.
- Week 9 (Jul 21–27): Implement `generation/validator.py` — post-generation `article_id` cross-check and `unverified` flagging, with unit tests.
- Week 10 (Jul 28–Aug 3): Implement `generation/pipeline.py` — end-to-end orchestration with off-topic guard. Build `evaluation/test_citations.py` — citation accuracy and false-positive flag rate.
- Week 11 (Aug 4–10): Build `ui/app.py` — Streamlit interface with document upload, compliance checklist, expandable citations, narrative summary, and

`st.session_state` follow-up support.
- Week 12 (Aug 11–17): Add stretch enhancements if ahead of schedule: hybrid BM25+dense search via `EnsembleRetriever`, `bge-reranker-base` reranking.
- Week 13 (Aug 18–24): Write reproducibility guide, clean up documentation, prepare final presentation slides.

Success Criteria: We consider citation validation successful if the false-positive flag rate is below 10% on the test DUL set.

Final deliverables (August 24): Complete end-to-end pipeline, evaluation reports for both retrieval and generation, and a reproducibility guide enabling a new REWS contributor to set up the system in under 30 minutes.

August 24 – November 2, 2026 (Extended Coding Period, if applicable)

If an extended timeline is granted, this period will be used to implement and benchmark all remaining stretch goals against the core pipeline baseline. Final deliverables will include an updated evaluation report, a blog post summarising the full project, and a packaged release on GitHub with version-tagged commits.

November 2, 2026 (Final Submission)

All code committed and tagged on GitHub. Repository includes ingestion scripts, populated vector store instructions, evaluation test harnesses with labelled data, Streamlit UI, full documentation, and a blog post summarising the project. All work traceable via Git commit history.

Minimum viable system (guaranteed regardless of timeline): Even under significant time pressure, the project will deliver a working end-to-end pipeline: ingestion of the GA4GH corpus with `article_id` metadata, MMR-based retrieval with `doc_type` filtering, basic clause-level verdict output with citation grounding, and a functional Streamlit UI. This baseline is complete by the end of Milestone 2. Hybrid search, the reranker, sub-query decomposition, and persistent session storage are only added after this baseline is stable and passing evaluation thresholds.

V. Future Improvements

Two natural extensions stand out beyond the GSoC timeline. The most valuable would be a DUO-aware reasoning layer that maps user document language directly to DUO ontology terms, enabling machine-readable compliance verdicts that downstream systems could consume programmatically. A second extension is a diff-based re-review workflow: when a researcher revises their DUL in response to a flagged gap, the system re-runs only the affected clauses rather than the full pipeline, reducing turnaround time for iterative compliance review. A third extension is OCR support for scanned documents via PyTesseract, which would allow researchers to upload image-based or legacy consent forms that are not available in machine-readable format.

VI. Continued Involvement

I intend to remain involved with GA4GH's REWS workstream after GSoC by maintaining RegBot as a community tool. I plan to update the corpus as new GA4GH frameworks are published and triaging feedback from researchers who use it. In the longer term, I am

interested in helping design evaluation standards for AI-assisted compliance tools within the GA4GH ecosystem, which I see as a natural evolution of the work this project begins.

VII. Conflict of Interests or Commitments

I have no internship, fellowship, or full-time employment commitments during the GSoC period that would compete for my time or restrict my ability to assign copyright of code developed during the programme to GA4GH. I am able to commit approximately 30–35 hours per week to this project across the standard 12-week coding period, which I believe is sufficient to complete both the core deliverables and make meaningful progress on stretch goals.

I am currently in the United States on an F-1 student visa. I am aware that F-1 visa holders accepted as GSoC contributors must obtain appropriate authorisation before participation. I am actively working with Bucknell's international student office to confirm the necessary approvals are in place prior to the coding start date of May 25, 2026, and will communicate the status of this process to my mentor and the GA4GH org administrators promptly upon acceptance.

VIII. Major Challenges Foreseen

Clause boundary detection during ingestion. GA4GH policy documents do not use a consistent heading structure across frameworks. The `article_id` regex in `chunker.py` may miss clauses in documents that use non-standard numbering or prose-embedded references. Mitigation: the hand-labelled evaluation set will be used to measure extraction recall before the ingestion pipeline is finalised, and fallback heuristics (section heading detection, paragraph boundary rules) will be added if the regex alone is insufficient.

Structured JSON from small LLMs. Llama 3 8B and Mistral 7B sometimes produce malformed JSON when asked for structured output, particularly under complex system prompts. Mitigation: the `validator.py` module already wraps JSON parsing in a try/except with a fallback to raw text, and the system prompt will use few-shot examples of correctly formatted verdict tables to improve reliability. If output consistency remains problematic, `outlines` or LangChain's `PydanticOutputParser` will be used to constrain generation.

Corpus coverage gaps. Some GA4GH guidance exists only as web pages or working group notes rather than stable PDFs. If a researcher's DUL references obligations from a framework that is not in the corpus, RegBot will produce a false `missing` verdict. Mitigation: the corpus will be documented explicitly so users know which frameworks are covered, and the UI will display a visible corpus coverage notice. Adding new documents requires only re-running `ingest.py`.

Building a ground truth evaluation set. Building a reliable hand-labelled test set of clause lookups requires policy expertise. Mitigation: I will use the publicly available GA4GH framework documents and construct test cases from the clause structure directly, then ask my mentor to review a sample for correctness before the evaluation is run.

IX. References

- GA4GH Framework for Responsible Sharing of Genomic and Health-Related Data — https://www.ga4gh.org/genomic-data-toolkit/regulatory-ethics-toolkit/framework-for-

responsible-sharing-of-genomic-and-health-related-data/
- GA4GH Data Use Ontology (DUO) — https://github.com/EBISPOT/DUO
- GA4GH Consent Clarity Toolkit — https://www.ga4gh.org/genomic-data-toolkit/regulatory-ethics-toolkit/consent-toolkit/
- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — https://arxiv.org/abs/2005.11401
- Barnett et al. (2024). Seven Failure Points When Engineering a Retrieval Augmented Generation System — https://arxiv.org/abs/2401.05856
- Bhavsar, P. (2024). Mastering RAG: How to Architect an Enterprise RAG System. Galileo. https://galileo.ai/blog/mastering-rag-how-to-architect-an-enterprise-rag-system
- LangChain documentation — https://python.langchain.com
- ChromaDB documentation — https://docs.trychroma.com
- BAAI/bge-reranker-base — https://huggingface.co/BAAI/bge-reranker-base
- Ollama — https://ollama.com

X. Relevant Background Experience

The main reason I think I can pull this off is that I have already built something very similar. OIRA-Chatbot is a full-stack academic assistant for Bucknell University that answers student questions about course requirements and academic policies. The system ingests institutional PDFs and Google Docs (course catalogs and academic policies), chunks and embeds them into ChromaDB with structured `doc_type` metadata, and uses MMR retrieval with metadata filtering to retrieve clause-precise context before generating grounded answers via GPT-4.1-mini.

The design decisions in that system directly map to RegBot's requirements. I implemented citation tracking as a first-class feature: every assistant message stores its citations as structured JSON, and the frontend renders expandable source references. I built a question classifier that routes queries to domain-specific retrieval filters, equivalent to RegBot's framework domain classifier. I implemented session-aware conversation context, a message regeneration workflow for iterative review, and a thumbs-up/down feedback collection system, all directly applicable to compliance review workflows where researchers ask follow-up questions and refine their submissions. The hardest problem I solved was ensuring the model only cited what was actually retrieved rather than reasoning from general knowledge about university policy, which I addressed through a strict system prompt grounding contract. RegBot's citation validator is a more rigorous version of that same idea, extended to cross-check `article_id` values programmatically. Understanding the full stack from document ingestion to user-facing UI means I can deliver a usable research tool rather than a notebook prototype.

XI. Personal Information

I am a third-year undergraduate student at Bucknell University in Lewisburg, Pennsylvania, pursuing a Bachelor of Science in Computer Science with a co-major in Data Science, expected graduation in May 2027. I maintain a 3.39 GPA and am a Dean's Scholarship recipient staying on the Dean's List for multiple semesters.

I first encountered GA4GH through my research into open-source organisations participating in GSoC 2026. What actually got me interested was that citation accuracy, which most RAG

projects treat as optional, is genuinely non-negotiable here. Since most RAG projects I have seen treat citation accuracy as a nice-to-have, that alignment between the technical challenge I am most interested in and a problem that matters to researchers globally is what made this project stand out.

I currently work part-time as the full-stack developer of the OIRA-Chatbot, a production academic assistant deployed at Bucknell University that answers student questions about course requirements and academic policies. This is not a course project or a prototype, but it is a live system with future real users, persistent session history, feedback tracking, and a citation-grounded RAG pipeline backed by ChromaDB and LangChain. Building and maintaining that system has given me practical experience with exactly the failure modes that matter for RegBot: retrieval precision, hallucination prevention, and making citation traceability a first-class system feature rather than an afterthought. I will continue as a developer and maintainer after deployment, a role that adds minimal ongoing time commitment.

Free Software Experience / Contributions

I have contributed to open-source projects in the GA4GH ecosystem and adjacent infrastructure communities.

Repository	PR	Status	Description	Language
elixir-cloud-aai /foca	#256	Open	Added `MONGO_URI` environment variable support with precedence logic over individual connection variables, backward compatibility, and full test coverage	Python
elixir-cloud-aai /foca	#257	Open	Added structured INFO logs for each access-control setup step, modernized deprecated `importlib.resources` usage, and hardened CI test assertions	Python
ga4gh/tool-registry-service-schemas	#280	Open	Fixed BEARER security scheme to use native HTTP bearer authentication in the TRS API spec	YAML
container-registry/harbor-satellite	#328	**Merged**	Added Go Report Card badge to README to surface code quality metrics	Go

container-registry/harbor-satellite	#331	Open	Centralized password policy enforcement inside `HashPassword()`, introduced `PasswordPolicyError` type to distinguish policy violations (400) from hashing failures (500)	Go
fossasia/pslab-python	#274	Open	Added `MockHandler` class and `ScienceLab(mock=True)` mode enabling hardware-free testing without a physical PSLab device	Python

Working on FOCA specifically gave me early exposure to GA4GH microservice conventions, community review standards, and the kind of iterative feedback cycle I expect during GSoC.

Language and Tools

- Python — proficient: FastAPI, LangChain, ChromaDB, PyTorch, scikit-learn, matplotlib, seaborn
- TypeScript / JavaScript — proficient: production Next.js and React
- Java, C — intermediate: coursework/projects
- SQL + MongoDB — intermediate: SQLite and MongoDB schema design for information storage in different formats
- Go — familiar: open-source contributions (harbor-satellite) and online course
- Git — proficient: experience with Github/Gitlab via projects/internships

Related Research and Work Experience

- Software Developer, Academic Chatbot + KALMUS — Bucknell University (Oct 2025 – Present): Built the OIRA-Chatbot, a production RAG system with a Next.js frontend, FastAPI backend, LangChain + ChromaDB pipeline, citation tracking, session history, and user feedback collection, which is the direct technical predecessor to RegBot. Concurrently built the KALMUS Movie Barcode Generator frontend, integrating with Bucknell's HPC cluster via SLURM for long-running compute jobs, which gives me familiarity with research infrastructure workflows similar to those GA4GH tools are deployed in.
- Undergraduate Researcher — Bucknell University (Oct 2024 – May 2025): Trained a real-time PyTorch gesture classifier for a prosthetics control system, including embedded deployment on Arduino. Demonstrates ML development and validation experience beyond web applications.

Reference Links

- Personal website: https://minhpham1810.github.io/
- Resume: Khoa_Minh_Pham_Resume.pdf
- GitHub: https://github.com/minhpham1810
- LinkedIn: https://www.linkedin.com/in/khoaminhpham18
- OIRA-Chatbot: https://github.com/OIRA-Chatbot-Project/OIRA-Chatbot
- KALMUS: https://github.com/minhpham1810/kalmus_web